You said:

We operate an ecommerce platform. When a customer places an order, the warehouse team
needs to know which shipping box should be used. Each product has dimensions and
weight. Each box has internal dimensions, maximum weight capacity, and cost.
Your task is to design and build a small Django-based system that recommends the most
suitable box for an order.
this is the the task i have to complete
i am making a AI based software which will assist the user which Length breadh and height box to select according to item dimension that to be packed, for this i am using a LLM(gemini)
i will connect it using API this will recommend the box
feature I want in this project is
1 user can save the different box of different dimension in database
2 when user give dimension of item then our software should recommend the which one to be used to pack that item and that box should be from our database box
3 gemini should recommend which box to be selected of which dimension

assist me to build this by giving a efficient code
i using django framework to built this
so lets start building this

ChatGPT said:

Let's build your **AI-Based Shipping Box Recommendation System using Django and Gemini API** step by step. We'll build it as a proper project that you can demonstrate in your assignment or portfolio.

The key idea is that users save their available shipping boxes in a database. When a user enters an item's dimensions and weight, Gemini recommends the most suitable box **from those saved in the database**, rather than suggesting an imaginary box.

1. How our project will work



Step 1: Add shipping boxes

Save box length, breadth, height, maximum weight and cost.





Step 2: Store boxes in Django database

Maintain a reusable catalog of available boxes.





Step 3: Enter item dimensions

The user enters length, breadth, height and weight.





Step 4: Gemini AI recommendation

Gemini evaluates eligible database boxes and explains its selection.





Step 5: Display recommended box

Show the existing box's dimensions, cost and reason for selection.

Recommended technology stack

- **Backend:** Python and Django
- **Database:** SQLite for initial development; PostgreSQL can be used later
- **AI:** Google Gemini API
- **Frontend:** Django Templates, HTML and CSS
- **Integration:** Google's official Python SDK

One important engineering decision: **Django must validate the dimensions and weight before accepting Gemini's recommendation.** The AI can rank eligible boxes and explain its reasoning, but it must not be allowed to select a box that physically cannot fit the item or exceeds the weight limit.

2. Create the Django project

We'll build the project in small stages so you understand every part of the code.

Step 1 — Set up the environment

Open your terminal in the folder where you want to create the project.

```
# Create a virtual environment
python -m venv venv

# Activate on Windows
venv\Scripts\activate

# Install dependencies
pip install django google-genai python-dotenv

# Create the Django project
django-admin startproject config .

# Create the application
python manage.py startapp boxes
```

We will use Google's official `google-genai` SDK to connect to Gemini. Google APIs +1

Your initial folder structure will look like this:

box_recommender/
- config/
- boxes/
  - models.py
  - views.py
  - forms.py — create this file
  - urls.py — create this file

services.py — create this file
admin.py

manage.py

.env

Step 2 — Register the application

Open `config/settings.py` and add `boxes` to `INSTALLED_APPS`.

```
INSTALLED_APPS = [
    # Default Django apps
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Our application
    "boxes",
]
```

Keep the default SQLite database configuration for now. We can switch to PostgreSQL later without changing the overall architecture.

3. Create the database model

Our first model will represent the shipping boxes available in the warehouse.

Open `boxes/models.py` and replace its contents with this:

```
from django.db import models
from django.core.validators import MinValueValidator


class ShippingBox(models.Model):
    name = models.CharField(max_length=100, unique=True)
```

```
    length = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0.01)],
    )
    breadth = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0.01)],
    )
    height = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0.01)],
    )

    max_weight = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0.01)],
        help_text="Maximum supported weight in kg",
    )

    cost = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0)],
        help_text="Cost per box",
    )

    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["cost", "name"]

    def __str__(self):
        return self.name

    @property
```

```
    def volume(self):
        return self.length * self.breadth * self.height
```

Why this model is efficient

- `DecimalField` avoids unnecessary floating-point precision issues for dimensions, weights and prices.
- `is_active` lets you disable a box without deleting its historical record.
- `volume` provides a convenient calculated property without storing redundant data.
- The unique name prevents duplicate box names.

Important: All dimensions must use the same unit. We will use centimeters for length, breadth and height, and kilograms for weight.

4. Register boxes in Django Admin

Open `boxes/admin.py` :

```
from django.contrib import admin
from .models import ShippingBox


@admin.register(ShippingBox)
class ShippingBoxAdmin(admin.ModelAdmin):
    list_display = (
        "name",
        "length",
        "breadth",
        "height",
        "max_weight",
        "cost",
        "is_active",
    )

    list_filter = ("is_active",)
    search_fields = ("name",)
    list_editable = ("is_active",)
```

This allows you to add, update, search and deactivate boxes from the Django admin panel before we build the custom frontend.

Run the migrations:

```
python manage.py makemigrations boxes
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open `http://127.0.0.1:8000/admin/` and log in with your superuser credentials. You should see the **Shipping boxes** model.

Add a few sample boxes to test the system:

Box name	Dimensions (cm)	Max weight	Cost
Small Box	15 × 10 × 8	2 kg	₹20
Medium Box	25 × 20 × 15	5 kg	₹35
Large Box	40 × 30 × 25	10 kg	₹60

These are example values for testing, not actual packaging specifications.

5. How we will implement the Gemini recommendation

After the database is working, we'll build the recommendation service in `boxes/services.py`.

The important part is to filter boxes using deterministic Python logic first, including allowing the item to be rotated inside the box.

For example, an item measuring `12 × 8 × 5 cm` can fit in a box measuring `10 × 15 × 8 cm` because the item can be rotated.

The flow will be:

1. Retrieve active boxes from the database.
2. Filter out boxes that cannot fit the item in any orientation or exceed its weight limit.
3. Send only the eligible boxes and their costs to Gemini.
4. Ask Gemini to select the most suitable box from the provided database IDs.

5. Validate Gemini's selected ID against the eligible set before displaying the result.
6. If Gemini is unavailable, use a deterministic fallback that selects the lowest-cost eligible box.

This design means Gemini assists with the recommendation, while Python enforces the physical constraints.

Your first milestone

Complete the database and admin setup above first.

Once you've done that, tell me **"done"** or share any error you encounter. We'll then implement the dimension-validation logic and Gemini API integration, followed by the user-facing pages to add boxes and request recommendations.

You said:

lets build the next phase of dimension validation and gemini API itegration after building this we will test whether it is recommending properly or not

ChatGPT said:

Great. Now we'll build the core intelligence of your project: **dimension validation + Gemini API integration + a testable recommendation engine**.

We'll keep the responsibilities separate:

- **Django/Python:** Validate dimensions, check rotations, enforce weight limits and retrieve boxes from the database.
- **Gemini:** Rank the eligible boxes and explain why one is suitable.
- **Django/Python again:** Verify that Gemini selected an eligible database box before returning the result.

This is important because an LLM can make mistakes with measurements. We should never trust its recommendation without checking it in Python.

Phase 1 — Configure the Gemini API

Install the dependencies if you haven't already:

```
pip install google-genai python-dotenv
```

Get a Gemini API key from Google's official Google AI Studio ↗.

Create a `.env` file in the same directory as `manage.py`:

```
GEMINI_API_KEY=your_actual_api_key_here
GEMINI_MODEL=gemini-2.5-flash
```

Replace the example API key with your own. Keep `.env` private and never commit it to GitHub.

Add the following to your `.gitignore`:

```
.env
venv/
__pycache__/
db.sqlite3
```

We'll load the API key through Django settings rather than hardcoding it in Python.

Phase 2 — Load environment variables in Django

Open `config/settings.py` and add these imports near the top:

```
import os
from dotenv import load_dotenv

load_dotenv(BASE_DIR / ".env")
```

Make sure `BASE_DIR` is already defined above this code. Then add:

```
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-flash-latest")
```

The model name is configurable, so you can change it in `.env` without editing your application code.

Phase 3 — Build the dimension validation service

Create or open `boxes/services.py`.

This service will validate the input, check every possible item orientation, and retrieve only boxes that can safely contain the item.

```
from decimal import Decimal, InvalidOperation
from itertools import permutations

from .models import ShippingBox


DIMENSION_FIELDS = ("length", "breadth", "height")


class DimensionValidationError(ValueError):
    """Raised when item dimensions or weight are invalid."""


def validate_item_dimensions(length, breadth, height, weight):
    """Convert inputs to Decimal and validate positive measurements."""

    values = {}

    for field, raw_value in {
        "length": length,
        "breadth": breadth,
        "height": height,
        "weight": weight,
    }.items():
        try:
            value = Decimal(str(raw_value))
        except (InvalidOperation, ValueError, TypeError):
            raise DimensionValidationError(
                f"{field.capitalize()} must be a valid number."
            )

        if not value.is_finite() or value <= 0:
            raise DimensionValidationError(
                f"{field.capitalize()} must be greater than zero."
```

```
            )

        values[field] = value

    return values


def item_fits_box(item_dimensions, box):
    """
    Check whether an item fits in a box in at least one orientation.
    Dimensions are in cm.
    """

    box_dimensions = (
        box.length,
        box.breadth,
        box.height,
    )

    for orientation in permutations(item_dimensions):
        if all(
            item_dimension <= box_dimension
            for item_dimension, box_dimension
            in zip(orientation, box_dimensions)
        ):
            return True

    return False


def get_eligible_boxes(item_dimensions, item_weight):
    """
    Return active database boxes that fit the item
    and support its weight.
    """

    eligible_boxes = []

    boxes = ShippingBox.objects.filter(
        is_active=True,
        max_weight__gte=item_weight,
    ).order_by("cost", "id")
```

```
    for box in boxes:
        if item_fits_box(item_dimensions, box):
            eligible_boxes.append(box)

    return eligible_boxes
```

Why this validation is important

Suppose the item measures `12 × 8 × 5 cm` and weighs `1 kg`.

- A `10 × 15 × 8 cm` box weighing capacity `2 kg` is eligible because the item can be rotated.
- A `10 × 10 × 4 cm` box is not eligible because the item cannot fit in any orientation.
- A box with dimensions `20 × 20 × 20 cm` but a maximum weight of `0.5 kg` is also ineligible.

The code checks both physical fit and maximum weight before Gemini is called.

Note that this checks rigid rectangular items and boxes. It does not yet account for cushioning, fragile products, packaging clearance, or the ability to compress an item.

Phase 4 — Integrate Gemini with the eligible boxes

We'll use Gemini's structured JSON output so the model returns a box ID and a reason, rather than arbitrary text that we would need to parse. Google's SDK supports schema-constrained responses through `generate_content`. Google APIs +1

Add the following code **below the existing functions** in `boxes/services.py`.

```
import json

from django.conf import settings
from google import genai


def recommend_box_with_gemini(
    item_dimensions,
    item_weight,
    eligible_boxes,
):
    """
```

```
    Ask Gemini to choose among eligible database boxes.

    Python remains responsible for validating the final selection.
    """

    if not eligible_boxes:
        return {
            "box": None,
            "reason": (
                "No available box can accommodate this item's "
                "dimensions and weight."
            ),
            "source": "validation",
        }

    # Deterministic fallback: cheapest eligible box.
    fallback_box = min(
        eligible_boxes,
        key=lambda box: (box.cost, box.pk),
    )

    api_key = getattr(settings, "GEMINI_API_KEY", None)

    if not api_key:
        return {
            "box": fallback_box,
            "reason": (
                "Gemini API key is not configured. "
                "Selected the lowest-cost eligible box instead."
            ),
            "source": "fallback",
        }

    # Send only boxes that Python has already validated.
    candidates = [
        {
            "id": box.pk,
            "name": box.name,
            "length_cm": float(box.length),
            "breadth_cm": float(box.breadth),
            "height_cm": float(box.height),
            "max_weight_kg": float(box.max_weight),
```

```
            "cost": float(box.cost),
        }
        for box in eligible_boxes
    ]

    prompt = f"""
You are a shipping packaging recommendation assistant.

Recommend the most suitable shipping box for the item.

Item:
- Length: {item_dimensions[0]} cm
- Breadth: {item_dimensions[1]} cm
- Height: {item_dimensions[2]} cm
- Weight: {item_weight} kg

Eligible boxes from our database:
{json.dumps(candidates)}

Rules:
1. Select exactly one box using its existing database ID.
2. Choose the lowest-cost suitable box when multiple options
   are equally suitable.
3. Prefer less unused space when cost is equal.
4. Never invent a box or database ID.
5. Explain your choice briefly.
"""

    try:
        client = genai.Client(api_key=api_key)

        response = client.models.generate_content(
            model=getattr(
                settings,
                "GEMINI_MODEL",
                "gemini-flash-latest",
            ),
            contents=prompt,
            config={
                "response_mime_type": "application/json",
                "response_json_schema": {
                    "type": "OBJECT",
```

```
                    "properties": {
                        "box_id": {"type": "INTEGER"},
                        "reason": {"type": "STRING"},
                    },
                    "required": ["box_id", "reason"],
                },
            },
        )

        if not response.text:
            raise ValueError("Gemini returned an empty response.")

        result = json.loads(response.text)
        selected_id = result.get("box_id")
        reason = result.get("reason")

        # Never trust an ID merely because Gemini returned it.
        selected_box = next(
            (
                box
                for box in eligible_boxes
                if box.pk == selected_id
            ),
            None,
        )

        if selected_box is None:
            raise ValueError(
                "Gemini selected an ID outside the eligible boxes."
            )

        if not isinstance(reason, str) or not reason.strip():
            raise ValueError("Gemini returned no explanation.")

        return {
            "box": selected_box,
            "reason": reason.strip(),
            "source": "gemini",
        }

    except Exception:
        # The system remains usable if the API fails.
```

```
        return {
            "box": fallback_box,
            "reason": (
                "Gemini recommendation was unavailable or invalid. "
                "Selected the lowest-cost eligible box instead."
            ),
            "source": "fallback",
        }
```

Why we use a fallback

The recommendation should still work if the API key is missing, Gemini times out, the quota is exhausted, or the model returns an invalid result.

The `source` field lets us distinguish an actual Gemini recommendation from a deterministic fallback during testing.

One thing to remember: the fallback guarantees the cheapest eligible box, not necessarily the globally optimal packaging choice. The recommendation objective can be improved later using packaging clearance, volume utilization and shipping costs.

Phase 5 — Create one function that combines everything

Still in `boxes/services.py`, add:

```
def recommend_shipping_box(length, breadth, height, weight):
    """
    Complete pipeline:
    input validation -> eligible boxes -> AI recommendation.
    """

    values = validate_item_dimensions(
        length,
        breadth,
        height,
        weight,
    )

    item_dimensions = (
        values["length"],
```

```
        values["breadth"],
        values["height"],
    )
    item_weight = values["weight"]

    eligible_boxes = get_eligible_boxes(
        item_dimensions,
        item_weight,
    )

    result = recommend_box_with_gemini(
        item_dimensions,
        item_weight,
        eligible_boxes,
    )

    result["eligible_count"] = len(eligible_boxes)

    return result
```

Your core recommendation engine is now ready. Any Django view, form or automated test can call `recommend_shipping_box()` instead of duplicating the logic.

Phase 6 — Create a Django endpoint to test recommendations

Before building the frontend, let's expose the engine through a Django endpoint. This makes it easier to test the recommendation with different dimensions.

Open `boxes/views.py`:

```
import json

from django.http import JsonResponse
from django.views.decorators.http import require_POST

from .services import (
    DimensionValidationError,
    recommend_shipping_box,
)
```

```
@require_POST
def recommend_box_view(request):
    try:
        data = json.loads(request.body)

        result = recommend_shipping_box(
            length=data.get("length"),
            breadth=data.get("breadth"),
            height=data.get("height"),
            weight=data.get("weight"),
        )

        box = result["box"]

        response_data = {
            "success": box is not None,
            "source": result["source"],
            "reason": result["reason"],
            "eligible_count": result["eligible_count"],
            "recommended_box": None,
        }

        if box:
            response_data["recommended_box"] = {
                "id": box.pk,
                "name": box.name,
                "length": str(box.length),
                "breadth": str(box.breadth),
                "height": str(box.height),
                "max_weight": str(box.max_weight),
                "cost": str(box.cost),
            }

        return JsonResponse(response_data)

    except DimensionValidationError as exc:
        return JsonResponse(
            {"success": False, "error": str(exc)},
            status=400,
        )
```

```
    except (json.JSONDecodeError, UnicodeDecodeError):
        return JsonResponse(
            {"success": False, "error": "Invalid JSON request."},
            status=400,
        )

    except (TypeError, AttributeError):
        return JsonResponse(
            {"success": False, "error": "Invalid request data."},
            status=400,
        )
```

This endpoint accepts JSON and returns the selected box, its dimensions, the cost, the explanation and the recommendation source.

Create `boxes/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path(
        "recommend/",
        views.recommend_box_view,
        name="recommend_box",
    ),
]
```

Now open `config/urls.py` and include the application URLs:

```
from django.contrib import admin
from django.urls import path, include

  urlpatterns = [
      path("admin/", admin.site.urls),
      path("api/", include("boxes.urls")),
  ]
```

Your endpoint will be:

```
POST http://127.0.0.1:8000/api/recommend/
```

It accepts JSON like this:

```
{
    "length": 12,
    "breadth": 8,
    "height": 5,
    "weight": 1
}
```

Phase 7 — Test the recommendation engine

First, ensure you have added the three sample boxes in Django Admin. Start the server:

```
python manage.py runserver
```

You can test directly from the Django shell without needing a frontend.

Run:

```
python manage.py shell
```

Then enter:

```
from boxes.services import recommend_shipping_box

result = recommend_shipping_box(
    length=12,
    breadth=8,
    height=5,
    weight=1,
)

print("Recommended box:", result["box"])
print("Reason:", result["reason"])
```

```
print("Source:", result["source"])
print("Eligible boxes:", result["eligible_count"])
```

Expected behavior with the sample database:

- Small Box (`15 × 10 × 8 cm` , capacity 2 kg) should be eligible.
- Medium Box and Large Box should also be eligible.
- Small Box costs the least, so it is the expected recommendation when all three sample boxes are available.

If the API key is configured and Gemini succeeds, `source` should be `"gemini"` . If the API is unavailable, `source` will be `"fallback"` .

Phase 8 — Test different scenarios

Let's test more than one item to verify that the recommendation changes correctly when the dimensions or weight change.

Recommendation test input

Use centimeters for dimensions and kilograms for weight. This is a local input preview; submit it to your Django endpoint to run the actual recommendation.

Length (cm)	Breadth (cm)
12	8
Height (cm)	**Weight (kg)**
5	1

Copy JSON test input

Copy the JSON, then use it in the request below.

You can call the endpoint from a Python shell using Django's test client. Django supports JSON POST requests directly through its test client. Django 6.2.d... +1

```
import json
from django.test import Client
```

```
client = Client()

payload = {
    "length": 12,
    "breadth": 8,
    "height": 5,
    "weight": 1,
}

response = client.post(
    "/api/recommend/",
    data=json.dumps(payload),
    content_type="application/json",
)

print(response.status_code)
print(response.json())
```

Run this inside `python manage.py shell` after saving the files and adding your sample boxes.

Test cases to verify

Test	Input	Expected behavior
Small item	12 × 8 × 5 cm, 1 kg	Small Box should be recommended
Medium item	22 × 18 × 12 cm, 3 kg	Medium Box should be recommended
Large item	35 × 25 × 20 cm, 8 kg	Large Box should be recommended
Excessive weight	12 × 8 × 5 cm, 15 kg	No eligible box
Oversized item	100 × 100 × 100 cm, 1 kg	No eligible box
Invalid dimensions	Length = -5 cm	HTTP 400 validation error

These expectations assume the three sample boxes from our earlier setup and no additional eligible boxes. The tests should also verify that rotation works and that inactive boxes are never recommended.

Phase 9 — Check the result

For the first test, a successful response should resemble this:

```
{
    "success": true,
    "source": "gemini",
    "reason": "The Small Box fits the item and has the lowest cost.",
    "eligible_count": 3,
    "recommended_box": {
        "id": 1,
        "name": "Small Box",
        "length": "15.00",
        "breadth": "10.00",
        "height": "8.00",
        "max_weight": "2.00",
        "cost": "20.00"
    }
}
```

The database ID and exact explanation may differ on your machine. This is an illustrative response, not a result from running your application.

Important improvement before production: The current view returns a generic error if the request body is not valid JSON, but the Gemini service also hides API exceptions to provide fallback behavior. During testing, add logging inside the service's `except` block so you can distinguish an invalid API key, quota error, network failure or invalid model name. Do not log your API key.

Once you've run the first test, share the actual response here, especially the `source`, `recommended_box` and `reason` fields. We can then debug the real recommendation behavior before moving on to the frontend.

You said:

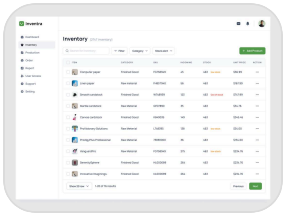
its recommending properly lets build the UI for the project

ChatGPT said:

Now that your dimension validation and Gemini recommendation are working, let's build the **user interface for your AI Shipping Box Recommendation System**.

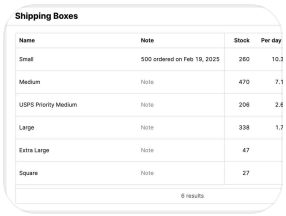
We'll use Django Templates, HTML and CSS, keeping your existing backend intact. The UI will have two main workflows: managing the warehouse's available boxes and recommending the best box for an item.

1. UI design and features



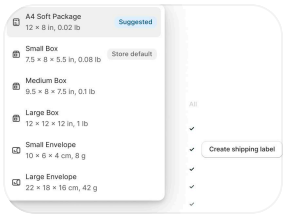
Dashboard

Overview of saved boxes, active inventory, and a quick link to get a recommendation.



Box management

Add boxes with dimensions, maximum weight, and cost. View, edit and deactivate existing boxes.



AI recommendation

Enter an item's dimensions and weight, then view the selected database box, price and Gemini's explanation.

We'll build a responsive dashboard with a navy-and-teal theme, clear measurement labels, and a result card that makes the recommended box easy for warehouse staff to identify.

2. Create the template and static-file structure

Inside your existing project, create these folders and files:

```
box_recommender/
├── config/
│   ├── settings.py
│   └── urls.py
```

```
├── boxes/
│   ├── forms.py
│   ├── views.py
│   ├── urls.py
│   ├── models.py
│   └── services.py
├── templates/
│   └── boxes/
│       ├── base.html
│       ├── dashboard.html
│       ├── box_list.html
│       ├── box_form.html
│       └── recommend.html
├── static/
│   └── css/
│       └── style.css
└── manage.py
```

At the project root, the `templates` and `static` directories should be alongside `manage.py`.

In `config/settings.py`, ensure these settings exist. Preserve your existing settings and add or update only the relevant parts.

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]

STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

3. Create Django forms

Create `boxes/forms.py`. These forms validate user input on the server, even if someone bypasses the HTML form.

```
from django import forms
from .models import ShippingBox
from .services import (
    DimensionValidationError,
    validate_item_dimensions,
)


class ShippingBoxForm(forms.ModelForm):
    class Meta:
        model = ShippingBox
        fields = [
            "name",
            "length",
            "breadth",
            "height",
            "max_weight",
            "cost",
            "is_active",
        ]

        widgets = {
            "name": forms.TextInput(attrs={
                "placeholder": "e.g. Medium Shipping Box",
            }),
            "length": forms.NumberInput(attrs={
                "step": "0.01", "min": "0.01",
            }),
            "breadth": forms.NumberInput(attrs={
                "step": "0.01", "min": "0.01",
            }),
            "height": forms.NumberInput(attrs={
                "step": "0.01", "min": "0.01",
            }),
            "max_weight": forms.NumberInput(attrs={
```

```
                "step": "0.01", "min": "0.01",
            }),
            "cost": forms.NumberInput(attrs={
                "step": "0.01", "min": "0",
            }),
        }


class RecommendationForm(forms.Form):
    length = forms.DecimalField(
        min_value=0.01,
        max_digits=10,
        decimal_places=2,
        label="Length (cm)",
    )
    breadth = forms.DecimalField(
        min_value=0.01,
        max_digits=10,
        decimal_places=2,
        label="Breadth (cm)",
    )
    height = forms.DecimalField(
        min_value=0.01,
        max_digits=10,
        decimal_places=2,
        label="Height (cm)",
    )
    weight = forms.DecimalField(
        min_value=0.01,
        max_digits=10,
        decimal_places=2,
        label="Item weight (kg)",
    )

    def clean(self):
        cleaned_data = super().clean()

        if all(
            field in cleaned_data
            for field in ("length", "breadth", "height", "weight")
        ):
            try:
```

```
                validate_item_dimensions(
                    cleaned_data["length"],
                    cleaned_data["breadth"],
                    cleaned_data["height"],
                    cleaned_data["weight"],
                )
            except DimensionValidationError as exc:
                self.add_error(None, str(exc))

        return cleaned_data
```

This gives us reusable validation for both the UI and the recommendation service.

4. Add the UI views

Your existing `recommend_box_view` API can remain unchanged. Add the following views to `boxes/views.py`, retaining the imports and endpoint code you already have.

```
from django.contrib import messages
from django.shortcuts import render, redirect, get_object_or_404
from django.views.decorators.http import require_POST

from .models import ShippingBox
from .forms import ShippingBoxForm, RecommendationForm
from .services import recommend_shipping_box


def dashboard(request):
    total_boxes = ShippingBox.objects.count()
    active_boxes = ShippingBox.objects.filter(is_active=True).count()

    context = {
        "total_boxes": total_boxes,
        "active_boxes": active_boxes,
        "inactive_boxes": total_boxes - active_boxes,
        "recent_boxes": ShippingBox.objects.order_by(
            "-created_at"
        )[:5],
    }
    return render(request, "boxes/dashboard.html", context)
```

```
def box_list(request):
    boxes = ShippingBox.objects.all()

    return render(request, "boxes/box_list.html", {
        "boxes": boxes,
    })


def box_create(request):
    form = ShippingBoxForm(request.POST or None)

    if request.method == "POST" and form.is_valid():
        form.save()
        messages.success(request, "Shipping box saved successfully.")
        return redirect("boxes:box_list")

    return render(request, "boxes/box_form.html", {
        "form": form,
        "page_title": "Add Shipping Box",
    })


def box_edit(request, pk):
    box = get_object_or_404(ShippingBox, pk=pk)
    form = ShippingBoxForm(
        request.POST or None,
        instance=box,
    )

    if request.method == "POST" and form.is_valid():
        form.save()
        messages.success(request, "Shipping box updated successfully.")
        return redirect("boxes:box_list")

    return render(request, "boxes/box_form.html", {
        "form": form,
        "page_title": "Edit Shipping Box",
    })
```

```
@require_POST
def box_deactivate(request, pk):
    box = get_object_or_404(ShippingBox, pk=pk)
    box.is_active = False
    box.save(update_fields=["is_active", "updated_at"])

    messages.success(request, f"{box.name} was deactivated.")
    return redirect("boxes:box_list")


def recommend_box_page(request):
    form = RecommendationForm(request.POST or None)
    result = None

    if request.method == "POST" and form.is_valid():
        data = form.cleaned_data

        result = recommend_shipping_box(
            length=data["length"],
            breadth=data["breadth"],
            height=data["height"],
            weight=data["weight"],
        )

    return render(request, "boxes/recommend.html", {
        "form": form,
        "result": result,
    })
```

Important: Keep your existing JSON API view and its imports in `views.py`. The new `recommend_box_page` is a separate HTML page that calls the same recommendation service.

5. Configure the page URLs

Replace `boxes/urls.py` with the following, including your existing JSON API route:

```
from django.urls import path
from . import views

app_name = "boxes"
```

```
urlpatterns = [
    path("", views.dashboard, name="dashboard"),
    path("boxes/", views.box_list, name="box_list"),
    path("boxes/add/", views.box_create, name="box_create"),
    path("boxes/<int:pk>/edit/", views.box_edit, name="box_edit"),
    path(
        "boxes/<int:pk>/deactivate/",
        views.box_deactivate,
        name="box_deactivate",
    ),
    path("recommend/", views.recommend_box_page, name="recommend_page"),

    # Existing JSON endpoint
    path("api/recommend/", views.recommend_box_view, name="recommend_api"),
]
```

Then update `config/urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("boxes.urls")),
]
```

The pages will now be available at:

Page	URL
Dashboard	`/`
Box inventory	`/boxes/`
Add a box	`/boxes/add/`
Edit a box	`/boxes/1/edit/`
AI recommendation	`/recommend/`

JSON API	`/api/recommend/`

6. Build the shared layout

Create `templates/boxes/base.html` . All the other pages will extend this file so the navigation and styling stay consistent.

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}PackSmart AI{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <aside class="sidebar">
        <a href="{% url 'boxes:dashboard' %}" class="brand">
            <span class="brand-icon">P</span>
            <span>PackSmart <small>AI PACKAGING</small></span>
        </a>

        <p class="nav-label">WORKSPACE</p>
        <a class="nav-link" href="{% url 'boxes:dashboard' %}">
            <span>▦</span> Dashboard
        </a>
        <a class="nav-link" href="{% url 'boxes:box_list' %}">
            <span>▣</span> Box Inventory
        </a>
        <a class="nav-link" href="{% url 'boxes:box_create' %}">
            <span>＋</span> Add New Box
        </a>

        <p class="nav-label">INTELLIGENCE</p>
        <a class="nav-link" href="{% url 'boxes:recommend_page' %}">
            <span>✳</span> AI Recommendation
        </a>

        <div class="sidebar-footer">
```

```
            <span class="status-dot"></span>
            Packaging engine ready
            <small>Dimension validation enabled</small>
        </div>
    </aside>

    <main class="main">
        <header class="topbar">
            <div>
                <span class="eyebrow">WAREHOUSE OPERATIONS</span>
                <div class="topbar-title">Shipping Intelligence</div>
            </div>
            <a href="{% url 'boxes:recommend_page' %}" class="button button-primary">
                ✳ Get recommendation
            </a>
        </header>

        <section class="page-content">
            {% if messages %}
                {% for message in messages %}
                    <div class="message {{ message.tags }}">
                        {{ message }}
                    </div>
                {% endfor %}
            {% endif %}

            {% block content %}{% endblock %}
        </section>

        <footer class="footer">
            PackSmart AI · Smart packaging decisions
        </footer>
    </main>
</body>
</html>
```

7. Create the dashboard

Create `templates/boxes/dashboard.html`.

```
{% extends "boxes/base.html" %}

{% block title %}Dashboard | PackSmart AI{% endblock %}

{% block content %}
<div class="welcome-row">
    <div>
        <span class="eyebrow">OVERVIEW</span>
        <h1>Your packaging workspace</h1>
        <p class="muted">
            Manage shipping boxes and find the right fit for every order.
        </p>
    </div>
    <a href="{% url 'boxes:box_create' %}" class="button button-outline">
        + Add a box
    </a>
</div>

<div class="stats-grid">
    <div class="stat-card">
        <span class="stat-icon">▣</span>
        <p>Total boxes</p>
        <h2>{{ total_boxes }}</h2>
        <span class="muted">Saved in inventory</span>
    </div>
    <div class="stat-card">
        <span class="stat-icon green">✓</span>
        <p>Active boxes</p>
        <h2>{{ active_boxes }}</h2>
        <span class="muted">Available for recommendations</span>
    </div>
    <div class="stat-card">
        <span class="stat-icon orange">II</span>
        <p>Inactive boxes</p>
        <h2>{{ inactive_boxes }}</h2>
        <span class="muted">Not used for recommendations</span>
    </div>
</div>

<div class="hero-card">
```

```
        <div class="hero-copy">
            <span class="pill">AI-POWERED PACKAGING</span>
            <h2>Every item deserves<br>a perfectly fitted box.</h2>
            <p>
                Enter your item's dimensions and weight. Our validation
                engine checks the available boxes, while Gemini helps
                select a suitable option.
            </p>
            <a href="{% url 'boxes:recommend_page' %}"
               class="button button-light">
                Recommend a box →
            </a>
        </div>
        <div class="hero-illustration" aria-hidden="true">
            <div class="box-back"></div>
            <div class="box-front">PACK<br>SMART</div>
            <div class="box-top"></div>
        </div>
    </div>

    <div class="section-heading">
        <div>
            <h2>Recently added boxes</h2>
            <p class="muted">Your latest inventory updates</p>
        </div>
        <a href="{% url 'boxes:box_list' %}" class="text-link">View all →</a>
    </div>

    <div class="table-card">
        <table>
            <thead>
                <tr>
                    <th>Box name</th>
                    <th>Dimensions (cm)</th>
                    <th>Max weight</th>
                    <th>Cost</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                {% for box in recent_boxes %}
                <tr>
```

```
                <td><strong>{{ box.name }}</strong></td>
                <td>{{ box.length }} × {{ box.breadth }} × {{ box.height }}</td>
                <td>{{ box.max_weight }} kg</td>
                <td>₹{{ box.cost }}</td>
                <td>
                    {% if box.is_active %}
                        <span class="badge badge-active">Active</span>
                    {% else %}
                        <span class="badge badge-inactive">Inactive</span>
                    {% endif %}
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="5" class="empty-state">
                    No boxes yet. Add your first shipping box to get started.
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}
```

8. Build the box inventory page

Create `templates/boxes/box_list.html`.

This page lets warehouse staff review the available catalog, edit box specifications, and deactivate boxes that are no longer available.

```
{% extends "boxes/base.html" %}

{% block title %}Box Inventory | PackSmart AI{% endblock %}

{% block content %}
<div class="welcome-row">
    <div>
        <span class="eyebrow">INVENTORY MANAGEMENT</span>
        <h1>Shipping boxes</h1>
```

```
            <p class="muted">Manage dimensions, capacities and packaging costs.</p>
        </div>
        <a href="{% url 'boxes:box_create' %}" class="button button-primary">
            + Add shipping box
        </a>
    </div>

    <div class="table-card">
        <table>
            <thead>
                <tr>
                    <th>Box name</th>
                    <th>Length</th>
                    <th>Breadth</th>
                    <th>Height</th>
                    <th>Capacity</th>
                    <th>Cost</th>
                    <th>Status</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for box in boxes %}
                <tr>
                    <td><strong>{{ box.name }}</strong></td>
                    <td>{{ box.length }} cm</td>
                    <td>{{ box.breadth }} cm</td>
                    <td>{{ box.height }} cm</td>
                    <td>{{ box.max_weight }} kg</td>
                    <td>₹{{ box.cost }}</td>
                    <td>
                        {% if box.is_active %}
                            <span class="badge badge-active">Active</span>
                        {% else %}
                            <span class="badge badge-inactive">Inactive</span>
                        {% endif %}
                    </td>
                    <td>
                        <div class="actions">
                            <a href="{% url 'boxes:box_edit' box.pk %}"
                               class="text-link">Edit</a>
```

```
                        {% if box.is_active %}
                        <form method="post"
                              action="{% url 'boxes:box_deactivate' box.pk %}">
                            {% csrf_token %}
                            <button type="submit" class="link-danger">
                                Deactivate
                            </button>
                        </form>
                        {% endif %}
                    </div>
                </td>
            </tr>
            {% empty %}
            <tr>
                <td colspan="8" class="empty-state">
                    No shipping boxes have been added yet.
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}
```

9. Create the add/edit box form

Create `templates/boxes/box_form.html`.

```
{% extends "boxes/base.html" %}

{% block title %}{{ page_title }} | PackSmart AI{% endblock %}

{% block content %}
<div class="welcome-row">
    <div>
        <span class="eyebrow">INVENTORY MANAGEMENT</span>
        <h1>{{ page_title }}</h1>
        <p class="muted">
            Enter the internal box dimensions and maximum weight capacity.
        </p>
```

```
        </div>
        <a href="{% url 'boxes:box_list' %}" class="button button-outline">
            ← Back to inventory
        </a>
    </div>

    <div class="form-card">
        <form method="post">
            {% csrf_token %}

            {% if form.non_field_errors %}
                <div class="form-errors">{{ form.non_field_errors }}</div>
            {% endif %}

            <h3>Box information</h3>
            <div class="form-grid">
                <div class="field full-width">
                    <label for="{{ form.name.id_for_label }}">Box name</label>
                    {{ form.name }}
                    {{ form.name.errors }}
                </div>

                <div class="field">
                    <label for="{{ form.length.id_for_label }}">Length (cm)</label>
                    {{ form.length }}
                    {{ form.length.errors }}
                </div>
                <div class="field">
                    <label for="{{ form.breadth.id_for_label }}">Breadth (cm)</label>
                    {{ form.breadth }}
                    {{ form.breadth.errors }}
                </div>
                <div class="field">
                    <label for="{{ form.height.id_for_label }}">Height (cm)</label>
                    {{ form.height }}
                    {{ form.height.errors }}
                </div>
                <div class="field">
                    <label for="{{ form.max_weight.id_for_label }}">
                        Maximum weight (kg)
                    </label>
                    {{ form.max_weight }}
```

```
                {{ form.max_weight.errors }}
            </div>
            <div class="field">
                <label for="{{ form.cost.id_for_label }}">
                    Cost per box (₹)
                </label>
                {{ form.cost }}
                {{ form.cost.errors }}
            </div>

            <div class="field full-width checkbox-field">
                {{ form.is_active }}
                <label for="{{ form.is_active.id_for_label }}">
                    Box is active and available for recommendations
                </label>
                {{ form.is_active.errors }}
            </div>
        </div>

        <div class="form-actions">
            <a href="{% url 'boxes:box_list' %}" class="button button-outline">
                Cancel
            </a>
            <button type="submit" class="button button-primary">
                Save shipping box
            </button>
        </div>
    </form>
</div>
{% endblock %}
```

10. Build the AI recommendation page

Create `templates/boxes/recommend.html`.

This is the primary page warehouse staff will use when an order arrives.

```
{% extends "boxes/base.html" %}

{% block title %}AI Recommendation | PackSmart AI{% endblock %}
```

```
{% block content %}
<div class="welcome-row">
    <div>
        <span class="eyebrow">SMART PACKAGING ASSISTANT</span>
        <h1>Find the right shipping box</h1>
        <p class="muted">
            Enter the item's dimensions and weight to find a suitable
            box from your saved inventory.
        </p>
    </div>
</div>

<div class="recommend-layout">
    <div class="form-card">
        <div class="card-heading">
            <span class="stat-icon">⌗</span>
            <div>
                <h2>Item specifications</h2>
                <p class="muted">Measurements are required in the units shown.</p>
            </div>
        </div>

        <form method="post">
            {% csrf_token %}

            {% if form.non_field_errors %}
                <div class="form-errors">{{ form.non_field_errors }}</div>
            {% endif %}

            <div class="form-grid">
                {% for field in form %}
                <div class="field">
                    <label for="{{ field.id_for_label }}">
                        {{ field.label }}
                    </label>
                    {{ field }}
                    {% if field.errors %}
                        <div class="field-error">{{ field.errors }}</div>
                    {% endif %}
                </div>
                {% endfor %}
```

```
            </div>

            <div class="info-note">
                <strong>How it works</strong>
                <p>
                    We validate the item against your active boxes,
                    check possible orientations and weight limits,
                    then ask Gemini to recommend a suitable option.
                </p>
            </div>

            <button type="submit" class="button button-primary button-wide">
                ✳ Find the best box
            </button>
        </form>
    </div>

    <div class="result-panel">
        {% if result %}
            {% if result.box %}
                <div class="result-success">
                    <span class="result-check">✓</span>
                    <span>RECOMMENDATION READY</span>
                </div>

                <h2>{{ result.box.name }}</h2>
                <p class="muted">
                    Selected from your saved shipping box inventory.
                </p>

                <div class="recommended-box">
                    <div class="box-symbol">▣</div>
                    <div>
                        <strong>{{ result.box.length }} ×
                            {{ result.box.breadth }} ×
                            {{ result.box.height }} cm</strong>
                        <p>Internal dimensions (L × B × H)</p>
                    </div>
                </div>

                <div class="result-metrics">
                    <div>
```

```
                    <span>Box cost</span>
                    <strong>₹{{ result.box.cost }}</strong>
                </div>
                <div>
                    <span>Max capacity</span>
                    <strong>{{ result.box.max_weight }} kg</strong>
                </div>
            </div>

            <div class="reason-box">
                <strong>Why this box?</strong>
                <p>{{ result.reason }}</p>
            </div>

            <div class="result-footer">
                <span>Recommendation source</span>
                {% if result.source == "gemini" %}
                    <span class="badge badge-ai">Gemini AI</span>
                {% elif result.source == "fallback" %}
                    <span class="badge badge-inactive">Rule-based fallback</span>
                {% else %}
                    <span class="badge">Validation</span>
                {% endif %}
            </div>

            <p class="muted small-text">
                {{ result.eligible_count }} eligible box(es) evaluated.
            </p>
        {% else %}
            <div class="no-result-icon">!</div>
            <h2>No suitable box found</h2>
            <p>{{ result.reason }}</p>
            <a href="{% url 'boxes:box_create' %}"
               class="button button-outline">
                Add a larger box
            </a>
        {% endif %}
    {% else %}
        <div class="empty-result">
            <div class="empty-result-icon">✳</div>
            <h2>Your recommendation will appear here</h2>
            <p>
```

```
                    Enter the item measurements and submit the form.
                    We'll check your inventory and display the selected box,
                    its cost and the reason for the recommendation.
                </p>
                <div class="mini-feature">✓ Physical fit validation</div>
                <div class="mini-feature">✓ Weight capacity checks</div>
                <div class="mini-feature">✓ Gemini-assisted selection</div>
            </div>
        {% endif %}
    </div>
</div>
{% endblock %}
```

11. Add the complete CSS styling

Create `static/css/style.css` and paste the following. This stylesheet provides the sidebar, dashboard cards, forms, tables, responsive layout and recommendation panel.

```
:root {
    --navy: #172b4d;
    --navy-light: #243d63;
    --teal: #0d9488;
    --teal-dark: #0f766e;
    --background: #f4f7fb;
    --surface: #ffffff;
    --text: #1e293b;
    --muted: #718096;
    --border: #e4eaf1;
    --green-bg: #e6f7ef;
    --red: #c24141;
    --radius: 14px;
    --shadow: 0 4px 20px rgba(27, 47, 75, 0.045);
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
```

```
    background: var(--background);
    color: var(--text);
    font-family: Inter, "Segoe UI", Arial, sans-serif;
    font-size: 14px;
    line-height: 1.6;
}

a {
    color: inherit;
    text-decoration: none;
}

h1, h2, h3, p {
    margin-top: 0;
}

h1 {
    margin-bottom: 7px;
    font-size: clamp(25px, 3vw, 32px);
    line-height: 1.25;
    letter-spacing: -0.8px;
}

h2 {
    font-size: 20px;
    line-height: 1.35;
}

h3 {
    font-size: 16px;
}

.sidebar {
    position: fixed;
    inset: 0 auto 0 0;
    width: 250px;
    padding: 27px 18px;
    background: var(--navy);
    color: #fff;
    display: flex;
    flex-direction: column;
    z-index: 10;
```

```
}

.brand {
    display: flex;
    align-items: center;
    gap: 12px;
    font-size: 18px;
    font-weight: 750;
    letter-spacing: -0.5px;
    margin-bottom: 45px;
}

.brand-icon {
    display: grid;
    place-items: center;
    width: 40px;
    height: 40px;
    border-radius: 12px;
    color: white;
    background: var(--teal);
    font-size: 23px;
}

.brand small {
    display: block;
    font-size: 9px;
    color: #a8bdd6;
    letter-spacing: 2px;
}

.nav-label {
    color: #8da4c2;
    font-size: 10px;
    font-weight: 750;
    letter-spacing: 1.5px;
    margin: 0 0 12px 10px;
}

.nav-link {
    display: flex;
    align-items: center;
    gap: 12px;
```

```
        padding: 12px;
        margin-bottom: 5px;
        color: #c6d3e4;
        border-radius: 9px;
        transition: background 0.2s, color 0.2s;
    }

    .nav-link:hover,
    .nav-link:focus-visible {
        background: var(--navy-light);
        color: white;
    }

    .nav-link span {
        width: 20px;
        text-align: center;
        font-size: 18px;
    }

    .nav-label:not(:first-child) {
        margin-top: 30px;
    }

    .sidebar-footer {
        margin-top: auto;
        padding: 17px 10px;
        border-top: 1px solid #344766;
        color: #d8e3ef;
        font-size: 12px;
    }

    .sidebar-footer small {
        display: block;
        margin-top: 5px;
        color: #8da4c2;
    }

    .status-dot {
        display: inline-block;
        width: 8px;
        height: 8px;
        border-radius: 50%;
```

```
        background: #4ade80;
        margin-right: 7px;
    }

    .main {
        min-height: 100vh;
        margin-left: 250px;
    }

    .topbar {
        min-height: 84px;
        padding: 16px 35px;
        display: flex;
        justify-content: space-between;
        align-items: center;
        gap: 15px;
        background: white;
        border-bottom: 1px solid var(--border);
    }

    .eyebrow {
        display: block;
        margin-bottom: 4px;
        color: var(--teal-dark);
        font-size: 10px;
        font-weight: 800;
        letter-spacing: 1.5px;
    }

    .topbar-title {
        font-size: 16px;
        font-weight: 700;
    }

    .page-content {
        max-width: 1500px;
        margin: auto;
        padding: 34px 35px;
    }

    .footer {
        padding: 20px 35px;
```

```
    color: var(--muted);
    border-top: 1px solid var(--border);
    font-size: 12px;
}

.welcome-row,
.section-heading {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 20px;
    margin-bottom: 25px;
}

.muted {
    color: var(--muted);
    margin-bottom: 0;
}

.button {
    display: inline-flex;
    justify-content: center;
    align-items: center;
    gap: 8px;
    padding: 11px 17px;
    border: 1px solid transparent;
    border-radius: 9px;
    font: inherit;
    font-weight: 650;
    cursor: pointer;
    transition: transform 0.15s, background 0.2s;
}

.button:hover {
    transform: translateY(-1px);
}

.button-primary {
    background: var(--teal);
    color: white;
}
```

```
.button-primary:hover {
    background: var(--teal-dark);
}

.button-outline {
    border-color: var(--border);
    background: white;
    color: var(--navy);
}

.button-light {
    background: white;
    color: var(--navy);
}

.button-wide {
    width: 100%;
    margin-top: 20px;
    padding: 14px;
}

.stats-grid {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 18px;
    margin-bottom: 25px;
}

.stat-card,
.form-card,
.table-card {
    padding: 23px;
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
}

.stat-card p {
    margin: 14px 0 0;
    color: var(--muted);
}
```

```
.stat-card h2 {
    margin: 2px 0 0;
    font-size: 31px;
    letter-spacing: -1px;
}

.stat-icon {
    display: inline-grid;
    place-items: center;
    width: 40px;
    height: 40px;
    border-radius: 11px;
    background: #e7efff;
    color: #335caa;
    font-size: 20px;
}

.stat-icon.green {
    color: #14845e;
    background: #e2f7ed;
}

.stat-icon.orange {
    color: #a96814;
    background: #fff0d9;
}

.hero-card {
    position: relative;
    overflow: hidden;
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 20px;
    min-height: 280px;
    padding: 35px;
    margin-bottom: 35px;
    border-radius: 18px;
    color: white;
    background: linear-gradient(120deg, #172b4d, #21486a 75%, #146d71);
}
```

```
.hero-copy {
    position: relative;
    z-index: 2;
    max-width: 570px;
}

.hero-copy h2 {
    margin: 18px 0 12px;
    font-size: clamp(25px, 3vw, 34px);
    letter-spacing: -0.8px;
}

.hero-copy p {
    max-width: 500px;
    margin-bottom: 22px;
    color: #d4e0ef;
}

.pill {
    display: inline-block;
    padding: 5px 10px;
    border: 1px solid #54748f;
    border-radius: 20px;
    color: #aaf1e7;
    font-size: 10px;
    font-weight: 750;
    letter-spacing: 1px;
}

.hero-illustration {
    position: relative;
    flex: 0 0 200px;
    height: 180px;
    margin-right: 30px;
    transform: rotate(-7deg);
}

.box-front {
    position: absolute;
    inset: 65px 25px 5px;
    display: grid;
```

```
        place-items: center;
        text-align: center;
        font-size: 20px;
        font-weight: 850;
        letter-spacing: 3px;
        line-height: 1.5;
        color: #16485a;
        background: linear-gradient(135deg, #e6bd80, #c99151);
        clip-path: polygon(0 0, 100% 0, 100% 100%, 0 100%);
    }

    .box-top {
        position: absolute;
        top: 24px;
        left: 25px;
        width: 150px;
        height: 70px;
        background: #f2d4a3;
        clip-path: polygon(50% 0, 100% 35%, 50% 70%, 0 35%);
    }

    .box-back {
        position: absolute;
        inset: 10px 20px 50px;
        background: #cda16b;
        opacity: 0.55;
        clip-path: polygon(50% 0, 100% 40%, 50% 80%, 0 40%);
    }

    .section-heading {
        margin-bottom: 15px;
    }

    .section-heading h2 {
        margin-bottom: 4px;
    }

    .text-link {
        color: var(--teal-dark);
        font-weight: 700;
    }
```

```
.table-card {
    overflow-x: auto;
    padding: 0;
}

table {
    width: 100%;
    border-collapse: collapse;
    white-space: nowrap;
}

th {
    padding: 15px 18px;
    text-align: left;
    color: #64748b;
    background: #f8fafc;
    font-size: 11px;
    letter-spacing: 0.4px;
    text-transform: uppercase;
}

td {
    padding: 17px 18px;
    border-top: 1px solid var(--border);
}

tbody tr:hover {
    background: #fbfdff;
}

.badge {
    display: inline-block;
    padding: 4px 9px;
    border-radius: 20px;
    background: #edf2f7;
    color: #526174;
    font-size: 11px;
    font-weight: 700;
}

.badge-active,
.badge-ai {
```

```
        background: var(--green-bg);
        color: #14784f;
    }

    .badge-inactive {
        background: #fff0e6;
        color: #a34f1c;
    }

    .badge-ai {
        background: #e0f2fe;
        color: #075985;
    }

    .empty-state {
        padding: 35px;
        text-align: center;
        color: var(--muted);
    }

    .actions {
        display: flex;
        align-items: center;
        gap: 13px;
    }

    .actions form {
        display: inline;
    }

    .link-danger {
        padding: 0;
        border: 0;
        background: transparent;
        color: var(--red);
        font: inherit;
        cursor: pointer;
    }

    .form-card {
        max-width: 900px;
    }
```

```
.form-card h3 {
    margin-bottom: 22px;
}

.form-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 20px;
}

.field {
    min-width: 0;
}

.field label {
    display: block;
    margin-bottom: 7px;
    font-weight: 650;
    color: #34445b;
}

.field input,
.field select,
.field textarea {
    width: 100%;
    min-height: 44px;
    padding: 10px 12px;
    border: 1px solid #d8e1eb;
    border-radius: 8px;
    outline: none;
    background: white;
    color: var(--text);
    font: inherit;
    transition: border-color 0.2s, box-shadow 0.2s;
}

.field input:focus,
.field select:focus,
.field textarea:focus {
    border-color: var(--teal);
    box-shadow: 0 0 0 3px rgba(13, 148, 136, 0.12);
```

```
}

.full-width {
    grid-column: 1 / -1;
}

.checkbox-field {
    display: flex;
    align-items: center;
    gap: 10px;
}

.checkbox-field input {
    width: 17px;
    min-height: 17px;
}

.checkbox-field label {
    margin: 0;
}

.form-actions {
    display: flex;
    justify-content: flex-end;
    gap: 12px;
    margin-top: 30px;
    padding-top: 22px;
    border-top: 1px solid var(--border);
}

.errorlist,
.field-error,
.form-errors {
    margin: 5px 0 0;
    color: #b42318;
    font-size: 12px;
}

.errorlist {
    padding-left: 17px;
}
```

```
.message {
    margin-bottom: 18px;
    padding: 13px 16px;
    border-radius: 9px;
    background: #e6f7ef;
    color: #166344;
}

.message.error {
    background: #feecec;
    color: #a12828;
}

.recommend-layout {
    display: grid;
    grid-template-columns: minmax(0, 1.1fr) minmax(0, 0.9fr);
    gap: 24px;
    align-items: start;
}

.recommend-layout .form-card {
    max-width: none;
}

.card-heading {
    display: flex;
    align-items: center;
    gap: 13px;
    margin-bottom: 27px;
}

.card-heading h2 {
    margin: 0 0 3px;
}

.card-heading p {
    font-size: 12px;
}

.info-note {
    margin-top: 22px;
    padding: 15px;
```

```
        border: 1px solid #d9e9f8;
        border-radius: 9px;
        background: #f0f7ff;
    }

    .info-note strong {
        color: #254d78;
    }

    .info-note p {
        margin: 5px 0 0;
        color: #536e89;
        font-size: 12px;
    }

    .result-panel {
        min-width: 0;
        padding: 26px;
        border: 1px solid var(--border);
        border-radius: var(--radius);
        background: white;
        box-shadow: var(--shadow);
    }

    .result-success {
        display: flex;
        align-items: center;
        gap: 9px;
        margin-bottom: 18px;
        color: var(--teal-dark);
        font-size: 10px;
        font-weight: 800;
        letter-spacing: 1px;
    }

    .result-check {
        display: grid;
        place-items: center;
        width: 26px;
        height: 26px;
        border-radius: 50%;
        background: var(--green-bg);
```

```
        font-size: 14px;
    }

    .result-panel h2 {
        margin-bottom: 7px;
        font-size: 25px;
    }

    .recommended-box {
        display: flex;
        align-items: center;
        gap: 16px;
        padding: 18px;
        margin: 23px 0;
        border: 1px solid #dcebe8;
        border-radius: 11px;
        background: #f1faf8;
    }

    .box-symbol {
        display: grid;
        place-items: center;
        flex: 0 0 54px;
        height: 54px;
        border-radius: 12px;
        background: #d8f2eb;
        color: var(--teal-dark);
        font-size: 28px;
    }

    .recommended-box strong {
        font-size: 15px;
    }

    .recommended-box p {
        margin: 4px 0 0;
        color: var(--muted);
        font-size: 12px;
    }

    .result-metrics {
        display: grid;
```

```
        grid-template-columns: repeat(2, minmax(0, 1fr));
        gap: 12px;
        margin-bottom: 22px;
    }

    .result-metrics div {
        padding: 15px;
        border: 1px solid var(--border);
        border-radius: 9px;
    }

    .result-metrics span,
    .result-footer > span:first-child {
        display: block;
        margin-bottom: 6px;
        color: var(--muted);
        font-size: 12px;
    }

    .result-metrics strong {
        font-size: 20px;
    }

    .reason-box {
        padding: 17px;
        border-radius: 9px;
        background: #f6f8fc;
    }

    .reason-box p {
        margin: 7px 0 0;
        color: #526174;
        line-height: 1.7;
    }

    .result-footer {
        display: flex;
        justify-content: space-between;
        align-items: center;
        gap: 12px;
        margin-top: 22px;
    }
```

```
.result-footer > span:first-child {
    margin-bottom: 0;
}

.small-text {
    margin-top: 14px;
    font-size: 11px;
}

.empty-result {
    padding: 30px 5px;
    text-align: center;
}

.empty-result-icon,
.no-result-icon {
    display: grid;
    place-items: center;
    width: 65px;
    height: 65px;
    margin: 0 auto 20px;
    border-radius: 18px;
    background: #e0f5f1;
    color: var(--teal-dark);
    font-size: 32px;
}

.empty-result h2 {
    max-width: 280px;
    margin: 0 auto 12px;
}

.empty-result > p {
    max-width: 330px;
    margin: 0 auto 25px;
    color: var(--muted);
}

.mini-feature {
    padding: 10px;
    margin-top: 8px;
```

```
        border-radius: 8px;
        background: #f7fafc;
        color: #40576f;
        text-align: left;
    }

    .no-result-icon {
        background: #fff0e6;
        color: #a34f1c;
    }

    @media (max-width: 1150px) {
        .sidebar {
            width: 215px;
        }

        .main {
            margin-left: 215px;
        }

        .page-content {
            padding: 26px 22px;
        }

        .recommend-layout {
            grid-template-columns: 1fr;
        }

        .hero-illustration {
            margin-right: 0;
        }
    }

    @media (max-width: 760px) {
        .sidebar {
            position: static;
            width: 100%;
            padding: 16px;
        }

        .brand {
            margin-bottom: 18px;
```

```
}

.nav-label,
.sidebar-footer {
    display: none;
}

.nav-link {
    display: inline-flex;
    padding: 9px;
    margin: 0 3px 4px 0;
    font-size: 12px;
}

.nav-label:not(:first-child) {
    margin-top: 10px;
}

.main {
    margin-left: 0;
}

.topbar {
    min-height: 72px;
    padding: 14px 18px;
}

.topbar .button {
    padding: 9px;
    font-size: 12px;
}

.page-content {
    padding: 23px 15px;
}

.footer {
    padding: 18px 15px;
}

.welcome-row {
    align-items: flex-start;
```

```
            flex-direction: column;
        }

        .stats-grid {
            grid-template-columns: 1fr;
            gap: 12px;
        }

        .stat-card {
            padding: 18px;
        }

        .hero-card {
            padding: 25px;
        }

        .hero-illustration {
            display: none;
        }

        .form-grid {
            grid-template-columns: 1fr;
        }

        .full-width {
            grid-column: auto;
        }

        .form-card {
            padding: 18px;
        }

        .result-panel {
            padding: 20px;
        }

        .section-heading {
            align-items: flex-start;
        }
    }
```

12. Run the project and verify the UI

Execute:

```
python manage.py check
python manage.py runserver
```

Open these pages in your browser:

Dashboard
http://127.0.0.1:8000/ ↗ Open

Box inventory
http://127.0.0.1:8000/boxes/ ↗ Open

AI recommendation
http://127.0.0.1:8000/recommend/ ↗ Open

The links will work when your local Django development server is running in the same browser environment.

Final checklist

- [ ] Dashboard shows the total, active and inactive box counts.
- [ ] Add and edit forms save records in your database.
- [ ] Deactivating a box removes it from future recommendations without deleting it.
- [ ] Invalid dimensions show form validation errors.
- [ ] The recommendation page displays the box returned by your existing service.
- [ ] Gemini and fallback recommendations are visibly distinguished.
- [ ] The layout works on mobile and desktop.

One security note: These pages currently assume a trusted, internal development environment. Before deploying the system, add user authentication and permissions so only authorized warehouse staff can modify inventory or trigger recommendations.

Once the pages are working, the next worthwhile improvement is to add recommendation history so you can see which item was packed into which box, the cost, and whether Gemini or the fallback selected it.